

LangGraph Cheatsheet

State, Node, Edge temelleri — ilk grafını kurmak için gereken her şey.

`pip install langgraph` · StateGraph / START / END · emineksknc.github.io/langgraph-turkce · **LangGraph v1.x** baz alınmıştır (Eylül 2026)

1. Neden LangGraph?

LCEL zinciri vs. Graf

LCEL (prompt | llm | parser) doğrusaldır. LangGraph; **döngü** (loop — aracı tekrar çağırma), **koşullu dallanma** (branching) ve **kalıcı state** (persistence) gerektiğinde kullanılır.

Workflow (iş akışı) vs Agent (ajan)

- **Workflow**: adım sırasını **kod** önceden belirler — öngörülebilir, test etmesi kolay
- **Agent**: adım sırasına **LLM** çalışma zamanında karar verir — esnek, test etmesi zor

Çoğu gerçek sistem ikisinin karışımıdır.

Üç temel yapı taşı

- **State** — tüm node'lar arasında akan paylaşılan veri
- **Node** — state alıp kısmi güncelleme döndüren fonksiyon
- **Edge** — node'lar arası akışı belirleyen bağlantı

2. State Tanımı

TypedDict + reducer

```
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages

class State(TypedDict):
    messages: Annotated[list, add_messages]
    kullanıcı_id: str
```

Hazır şablon: MessagesState

```
from langgraph.graph import MessagesState

class State(MessagesState):
    kullanıcı_id: str
```

Sadece mesaj geçmişi tutan state'ler için TypedDict yazmaya gerek bırakmaz.

Reducer (indirgeyici) neden gerekli?

Reducer yoksa her node state alanını **üzerine yazar**. `add_messages` yeni mesajları listeye **ekler** — mesaj geçmişi kaybolmaz.

Node — düğüm

Her **node** (düğüm), state alıp **kısmi güncelleme** döndüren bir fonksiyondur; tüm state'i değil, sadece değişen kısmı döndürür.

3. Node Fonksiyonu ve Grafı Kurma

Kısmi state döndür

```
def chatbot(state: State):
    yanıt = llm.invoke(state["messages"])
    return {"messages": [yanıt]}
```

Node, state'in **tamamını değil**, sadece değişen kısmı döndürür.

Grafı kur ve derle

```
from langgraph.graph import StateGraph, START, END

g = StateGraph(State)
g.add_node("chatbot", chatbot)
g.add_edge(START, "chatbot")
g.add_edge("chatbot", END)
app = g.compile()
app.invoke({"messages": [{"user": "merhaba"}]})
```

4. Koşullu Kenarlar (conditional edges / branching)

Karar fonksiyonu

```
def yon_bul(state: State) -> str:
    if state["messages"][-1].tool_calls:
        return "arac"
    return END

g.add_conditional_edges("chatbot", yon_bul)
```

path_map ile güvenli eşleme

```
g.add_conditional_edges(
    "chatbot", yon_bul,
    {"arac": "tools", "son": END})
```

`path_map` olmadan yazım hataları sessiz kalabilir.

5. Hızlı Referans

KAVRAM	NE İŞE YARAR
StateGraph	Graf tanımı oluşturur
START / END	Giriş / çıkış düğümleri
add_node	Grafa bir node (düğüm) ekler
add_edge	Sabit kenar (fixed edge) ekler
add_conditional_edges	Koşullu kenar (conditional edge) ekler
compile()	Grafı çalıştırılabilir hale getirir
invoke()	Grafı bir kez çalıştırır
MessagesState	Hazır mesaj-geçmiş state şablonu

Sık yapılan hata: Mesaj listesi gibi büyüyen state alanlarında reducer (add_messages) tanımlamamak — her node önceki geçmişi siler.

LangGraph Türkçe Rehber · Bağımsız, resmi olmayan eğitim kaynağı · LangGraph MIT lisanslıdır (langchain-ai) · Devamı: Orta Seviye Cheatsheet · Terim sözlüğü: emineksknc.github.io/langgraph-turkce/sozluk